Report

Shutterleaf

Barnabas Tanczos

8 April 2025

1 Overview

The objective of this practical was to design and implement a video game in the Processing framework, including character AI for navigating an environment and decision-making. All four levels of the requirements have been implemented in a modular game engine. The game *Shutterleaf* demonstrates these mechanics with various animals that interact with each other and the environment.

2 Design

The primary design goal of the *Shutterleaf* game was to showcase the various AI mechanics in the game engine, so that the player can observe these behaviours and explore their interactions in a relaxed environment. The result is an AI-focused simulation game with a single open forest map where various animals can move around and interact. Figure 1 shows a screenshot of the game in action. It has a pixel art aesthetic that captures the beauty of the natural environment in a simple, efficient manner. The forest map itself is randomly generated, offering a large space to explore. The animals that populate it have different behaviours for carnivores and herbivores, pursuing their own goals of nutrition and survival.

Rather than interacting with the animals directly, the player observes from an overhead perspective with no direct interaction. This encourages them to focus on the animals themselves, observing their behaviour and predicting their actions. However, they can influence the events indirectly by placing bait on the ground to lure animals into ideal positions. When the player takes a photo, they are incentivised to capture as many interesting elements as possible in the small frame. This provides a fun strategic element,



Figure 1: A screenshot from the *Shutterleaf* game, where the player captures a photo of a wolf hunting deer.

while remaining low-pressure as there is no loss state or limited resources. The game was inspired by wildlife simulation in games such as *New Pokémon Snap* (2021) and *Red Dead Redemption 2* (2018), as well as low-intensity, atmospheric games like *Abzû* (2016).

3 Implementation

3.1 Game Engine

The implementation of this game necessitated several updates to the game engine established in the first practical. A number of minor features were added, such as singular animations and new mouse input events. An important change was the addition of rotation in the `TransformComponent`. This represents the orientation of the object with an angle in radians normalized to the interval $(-\pi, \pi]$. Similarly to position, rotation is calculated from local rotation, which describes the difference in rotation from the parent. This allows child objects to automatically follow their parent's rotation. The component also enables conversion between rotation and vector direction using trigonometric identities, as well as turning the object towards a target.

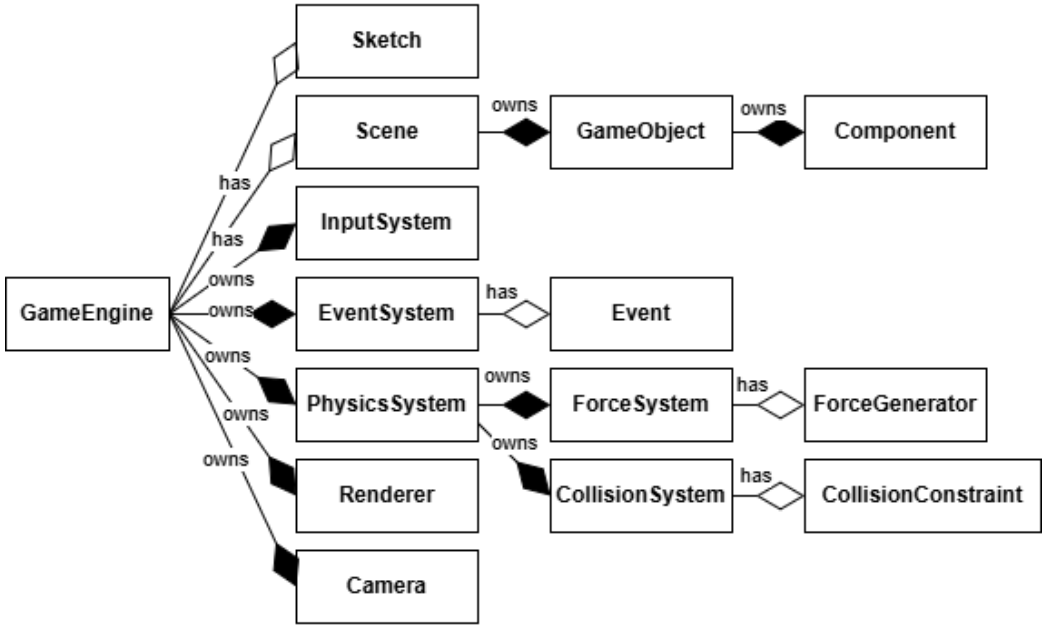


Figure 2: An updated UML diagram of the main relationships between the core game engine classes.

Two significant improvements were made to the core engine functionality, as seen in Figure 2. The first of these is the `Camera` class, which allows dynamic manipulation of which parts of the game world are displayed on screen. Instead of directly drawing to the `PApplet`, the camera stores the current frame visuals in the `camGraphics` buffer. This provides increased control and flexibility with the image, allowing transformations and effects to be applied exclusively to the camera view. The saved buffer snapshot is also necessary for the photography mechanics discussed later. At the beginning of each frame, the camera begins to capture the drawn graphics. All graphics have the same transformations applied to them to simulate the camera properties: a translation dependent on the camera position and a scaling based on the zoom. Finally, the capture ends before the end of the frame and the camera buffer is drawn over the screen. The `pushMatrix` and `popMatrix` methods keep the transformations from affecting further graphics. The class also provides methods to translate positions between the true world coordinates and the transformed screen coordinates.

The second major addition was the `Renderer` class. This was crucial for implementing the correct drawing order for the many sprites and background images on screen without relying solely on the order of the hierarchy. Therefore, instead of drawing visuals directly, components queue these ac-

tions using the `queueRender` method together with a layer parameter. After all components have updated, these graphics are then all drawn in a centralised manner by the `Renderer` in layer order. This ensures that they are all ordered correctly regardless of their origin. Rather than saving visuals in buffers, which would introduce a lot of overhead, the renderer instead stores them as `Runnable` functions that only need to be executed once. Furthermore, the `queueRenderOnTop` method provides UI rendering on top of all other graphics that is not affected by the camera.

3.2 AI Behaviours

A variety of AI behaviours were implemented in the engine using the modular component system, which could then be combined and added to objects. The first group of these rotate an object in set increments each frame towards a target: the direction of the velocity in `FaceAheadAI`, the direction of a point in `FaceTowardsAI`, or the rotation of another transform in `FaceAlignAI`. Similar simple scripts were created for kinematic movement. Using `KinematicSeekAI` and `KinematicFleeAI`, the object moves at a fixed speed towards or away from a given target transform, respectively. Meanwhile, `KinematicWanderAI` moves the object forward at a fixed speed while varying its rotation randomly, providing simple but not particularly realistic idle movement.

Further AI behaviours use force generators for better use of the physics engine and more compatibility between each other. The alternative `SeekAI` and `FleeAI`, for instance, apply cumulative acceleration in the target direction, allowing them to be combined with other forces. As these no longer immediately come to a halt when reaching their destination, they must cease accelerating before reaching it and account for the stop distance caused by drag. This is calculated using Newton's second law by solving the differential equation to get

$$\text{distance} = \frac{\text{mass} \times \text{velocity}}{\text{drag constant}}.$$

The object stops accelerating if it is within this radius or if it is within the target zone. The class `PursueAI` is a variant that moves towards the predicted position of the target instead of its current position. The prediction calculates the time to reach the target based on the object's velocity, and where the target would be after that time based on the target's velocity. Additionally, there is a more interesting implementation of random movement in `WanderAI`, which uses random pauses between bursts of movement.

The AI behaviours also provide ways to manage the distance between objects. With `SeparationAI`, a group of objects is forced to stay a given

minimum distance away from other members. Its complement, `CohesionAI`, draws a group of objects closer together by accelerating them towards the centroid, which is the mean position of the group. Together with `MatchAI`, which matches the velocities in a group, these behaviours can be combined to simulate flocking. Moreover, objects can avoid obstacles using `AvoidanceAI`, which guides them away at an angle from obstacles detected by an attached collider. For strictly bounding the range of movement, `BoundaryAI` can be used to push objects away from the edges if they get too close.

The most complex AI behaviour is the `PathfinderAI` based on Millington [1], which enables optimal navigation around walls and obstacles in a grid. This grid is represented by a two-dimensional array in `PathfindingGrid`, which performs localisation and quantisation by converting between grid coordinates and world positions. Cells can be removed from the grid to represent impassable obstacles. Given a start and end cell in the grid, a search algorithm can then be used to find the optimal path between them. This is implemented in the `Pathfinding` helper class using A* search. The algorithm keeps track of a frontier of states, which is expanded each iteration by calculating the possible next states until a goal state is reached. The frontier is a `PriorityQueue` sorted by the Euclidean distance to the goal and supplemented by an inability to go back to visited spaces. As this is an admissible heuristic, the calculated path is guaranteed to be optimal. Once a path is calculated, the `PathAI` moves object along the path. Rather than following the path exactly, it looks ahead to a point further along the path and moves towards it, creating a more natural motion.

3.3 Game

The game itself makes great use of these AI character behaviours for its main gameplay. All assets used are free public assets [2, 3, 4]. The forest map itself is created using simple random generation, as it would be too large to hand-place each element. It is created on a tile map that matches the cells of the pathfinding grid. Each tile is marked with an enum with a given probability for each: `GRASS` for regular empty grass tiles; `DECOR` for grass tiles with a decorative addition like a flower; `TREE` to mark where a tree will be placed; and `WATER` for water tiles. Tiles marked as water will also spread water in a small area around them to form ponds. The entire background image is then assembled, with particular care for the edges of the water tiles depending on the surrounding tiles. This image is saved to a buffer so it can be redrawn each frame without a significant rendering load. Trees are separate objects with collision and are drawn to different layers depending on their y coordinate. In this manner, as the layers of characters

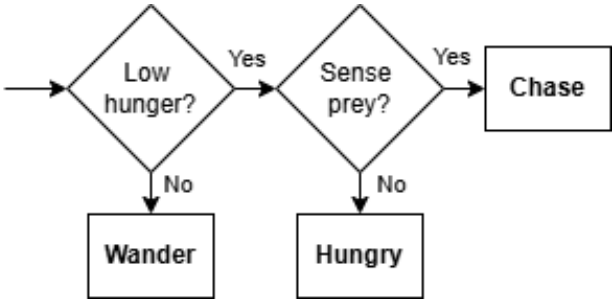


Figure 3: A diagram of the decision tree of a predator animal.

are also changed to reflect their y coordinate, they can appear behind trees or in front of them.

Using the new rendering and camera framework, the photo camera was straightforward to implement. When a photo is taken, a section of the previous camera buffer is retrieved and displayed on top of the world graphics using the UI rendering. The image is then scored based on the captured animals and environmental features, as detected by the camera collider.

Animals have one of two AI behaviours, with some different variables and movement parameters: `Predator` (wolf and fox) or `Prey` (deer and rabbit). Both types track their current behaviour state and animation state, which are decided based on their internal and external circumstances. These are given by tracked variables such as saturation, and senses that detect objects in a broad circle collider around them as well as a small collider directly in front of them. When the state of an animal changes, the corresponding AI behaviours are activated and deactivated to form the required new compound behaviour. Figure 3 shows the binary decision tree of a predator. This structure was preferred because the animal behaviour mainly has a strict order of priorities, as opposed to sequences of tasks more suitable for a state machine. A predator wanders around randomly until it gets hungry; then it uses pathfinding to begin purposefully searching for prey until it finds one and begins its chase. Figure 4 shows the more complex tree for prey. The first priority for these animals is to flee nearby predators. Only if it is safe does it begin to look for bait apples or, if it has a low saturation, a pond to drink.

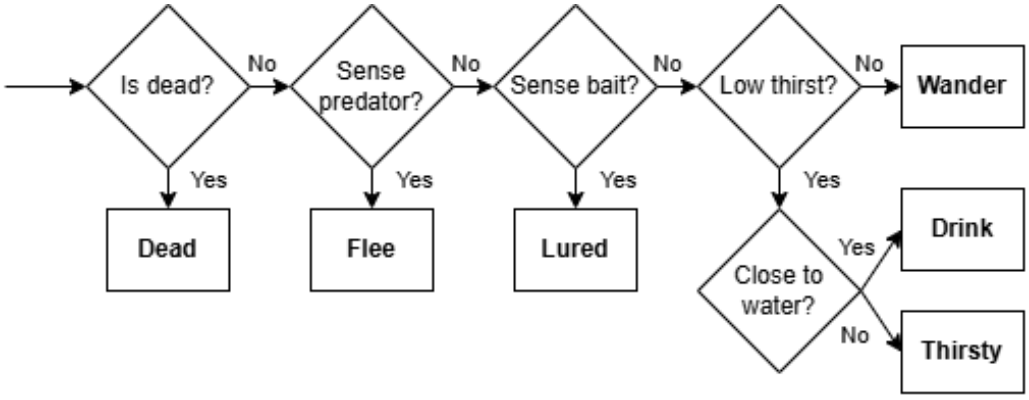


Figure 4: A diagram of the decision tree of a prey animal.

4 Evaluation

Overall, the designs of the new game engine features, including the AI behaviour framework, were a success. The new camera and rendering systems are a good step for more complex games that require precise ordering of drawn elements and a larger play space beyond a single screen. A significant variety of AI components were implemented, which can be reused in any project and combined to suit different needs. As a whole, they satisfy the project requirements well. Regarding the *Shutterleaf* game itself, I believe it achieved its goal of a relaxed experience with appealing, polished visuals and interesting behaviour patterns to observe. In my opinion, it has good potential to be extended into a replayable experience with more behaviour interactions and photography mechanics.

However, due to time constraints, the improvements to the engine and the number of AI components developed meant that the actual animal behaviours in the game don't meet the best standards for complexity or polish. The grid-based pathfinding does not always mesh well with other behaviours operating in a continuous plane. A complete AI behaviour specific to the game based entirely on pathfinding would have been more successful in avoiding clunky collisions and conflicting movements. Meanwhile, additional testing of the game would have fine-tuned the interactions to be more fair and believable. Moreover, quality-of-life additions such as sound effects and a saved picture catalogue would have helped create a more professional game feel. As the game engine framework improves with each project, I intend for the subsequent game to be more successful in this regard.

References

[1] Ian Millington. *AI for Games*. CRC Press, Boca Raton, US, 3rd edition, 2019.

[2] LYASeeK. MiniFolks - Forest Animals, 2023. Accessed: 2025-04-01 [online] https://lyaseek.itch.io/miniffanimals.

[3] GameToolkit.com. Top-Down Forest Tileset, 2021. Accessed: 2025-04-01 [online] https://gametoolkit.itch.io/top-down-forest-tileset.

[4] Roberto Mocci. Dogica font, 2020. Accessed: 2025-04-01 [online] https://www.dafont.com/dogica.font.